



Universidade de Brasília

FGA - Orientação a Objetos (FGA0158)

Daniel da Silveira Santos - 252030020

Juan Yury - 232027529

Miguel Tobias de Lima Galvão - 232005432

**DOCUMENTAÇÃO: SISTEMA DE VENDAS E FIDELIDADE — CAFETERIA GEEK
"BYTE & BREW"**

GAMA

2026

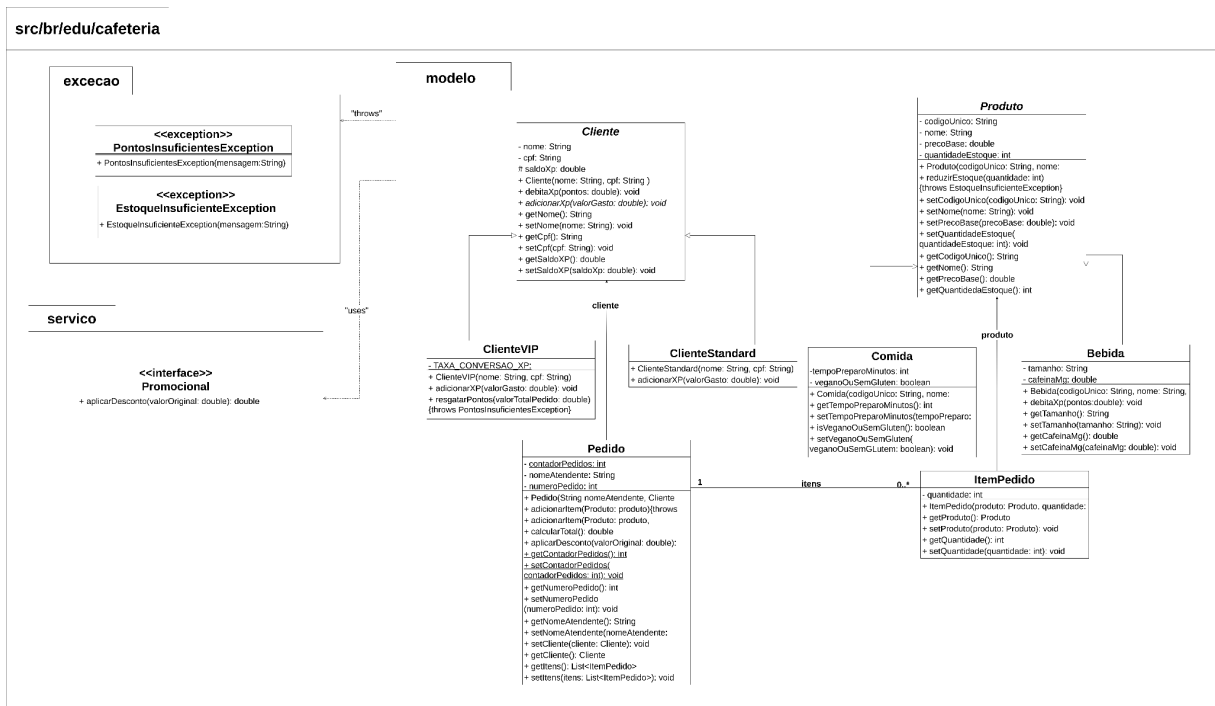
SUMÁRIO

SUMÁRIO.....	2
1. INTRODUÇÃO.....	3
2. DIAGRAMAS DE CLASSES UML.....	3
3. CONCEITOS DE ORIENTAÇÃO A OBJETOS APLICADOS.....	3
1. Associações.....	3
2. Herança Simples.....	4
3. Polimorfismo.....	4
4. Modificadores de escopo.....	4
5. Interfaces.....	4
6. Encapsulamento.....	4
4. TRATAMENTO DE EXCEÇÕES CUSTOMIZADAS.....	5
5. ESTRUTURA DE PACOTES.....	5

1. INTRODUÇÃO

O sistema Byte & Brew serve para automatizar e gerenciar as operações comerciais e o programa de relacionamento de uma cafeteria temática voltada para o público *geek*. O objetivo desta documentação técnica é detalhar como os requisitos de negócio foram traduzidos em uma estrutura de código utilizando os pilares do paradigma de Orientação a Objetos (OO).

2. DIAGRAMA DE CLASSES UML



Descrição geral das classes:

- **Produto:** Classe abstrata base para os itens do cardápio.
- **Comida e Bebida:** Subclasses que estendem *Produto* com suas especificidades.
- **Cliente:** Classe abstrata que representa o cliente cadastrado.
- **ClienteStandard e ClienteVIP:** Especializações da classe abstrata *Cliente*, seguindo as regras do cliente *Standard* e cliente *VIP*.
- **Pedido e ItemPedido:** Classes responsáveis pelo gerenciamento da venda e do carrinho de compras.

3. CONCEITOS DE ORIENTAÇÃO A OBJETOS APLICADOS

3.1. Associações

O sistema mapeia as interações de venda através das seguintes associações:

- *Pedido* e *ItemPedido*: Um *Pedido* gerencia uma lista de referências do tipo *ItemPedido*, por meio do atributo *itens*.
- *ItemPedido* e *Produto*: Cada item do pedido aponta para um *Produto* do cardápio para consultar valores e dar baixa no estoque.
- *Pedido* e *Cliente*: Um *Pedido* aponta para um *Cliente*, caso esteja registrado.

3.2. Herança Simples

- Produtos: *Comida* e *Bebida* herdam de *Produto*, compartilhando os atributos *codigoUnico*, *nome*, *precoBase* e *quantidadeEstoque*.
- Clientes: *ClienteStandard* e *ClienteVIP* herdam de *Cliente*, compartilhando os atributos *nome*, *cpf* e *saldoXP*.

3.3. Polimorfismo

- Polimorfismo por Inclusão: A classe *ItemPedido* armazena um atributo genérico do tipo *Produto*, podendo guardar uma referência tanto a classe *Comida* quanto *Bebida*. Na classe *Pedido*, existe um atributo do tipo *Cliente*, que pode referenciar tanto os subtipos *ClienteStandard* quanto *ClienteVIP*. Também possui uma lista de atributos do tipo *ItemPedido*, que por sua vez fazem o polimorfismo por inclusão mostrado anteriormente.
- Polimorfismo por Sobrescrita: O método *adicionarXP(double valorGasto)* foi sobrescrito nas subclasses de *Cliente*. Na classe *ClienteStandard*, é aplicado o fator de 1 ponto por real gasto, enquanto na *ClienteVIP* aplica-se o multiplicador de 2 pontos por real. A classe *Pedido* implementa o método *double aplicarDesconto(double valorOriginal)* da interface *Promocional*.
- Polimorfismo por Sobrecarga: Na classe *Pedido*, o método de inserção de itens foi sobrecarregado com duas assinaturas:
 1. *adicionarItem(Produto p)*: Insere por padrão a quantidade de 1 unidade.
 2. *adicionarItem(Produto p, int quantidade)*: Permite especificar a quantidade exata desejada.
- Polimorfismo por Coerção: Aplicado nas operações matemáticas do sistema. Há coerção implícita ao misturar variáveis do tipo *int* com tipo *double* (quantidade de itens * preço base) no cálculo de subtotais.

3.4. Modificadores de Escopo (Estático vs. Dinâmico)

Foram utilizados atributos estáticos onde necessário para manter um mesmo valor entre as diferentes instâncias das classes:

- Utilizado na classe *Pedido* (*private static int contadorSequencial; static int getContadorPedidos()*) para garantir que o número da comanda permaneça incrementado entre pedidos.
- Utilizado no fator de conversão de pontos XP na classe *ClienteVIP* (*private static final double TAXA_CONVERSAO = 10.0;*).

3.5. Interfaces

A interface *Promocional* foi utilizada para declarar o método utilizado para aplicar as promoções. Ela é implementada para a promoção do dia geek por meio do polimorfismo por sobrescrita, assim permitindo que as regras específicas de promoção do dia geek sejam implementadas.

3.6. Encapsulamento

Nenhum atributo de classe foi exposto publicamente. Todos os campos foram declarados como *private* ou *protected* (quando necessários para as subclasses), garantindo o isolamento dos dados e o acesso controlado através de métodos assessores (*getters* e *setters*) e construtores bem definidos.

4. TRATAMENTO DE EXCEÇÕES CUSTOMIZADAS

O projeto usa duas exceções customizadas:

- *EstoqueInsuficienteException* (Checked)
- *PontosInsuficientesException* (Checked)

As exceções do tipo checked foram escolhidas por representarem erros previsíveis no código que devem necessariamente ser tratados. Como as exceções do tipo checked forçam o código a tratá-las, foram a escolha mais adequada.

5. ESTRUTURA DE PACOTES

O código fonte foi separado nos seguintes pacotes:

- *br.edu.cafeteria.modelo*: Contém as classes de negócio (*Produto*, *Comida*, *Bebida*, *Cliente*, etc.).
- *br.edu.cafeteria.excecao*: Contém as exceções customizadas do sistema.
- *br.edu.cafeteria.servico*: Contém a interface de descontos.
- *br.edu.cafeteria.app*: Contém a classe Main que inicializa o sistema.